RETIREMENT PLANNER

GitHub Repository Setup, Cloning, Structure Generation, and Initial Population Guide

For retirement-planner-github.zip

Version 1.0 | July 20, 2026

1. Purpose and Recommended Method

This guide creates a separate GitHub repository for the Retirement Planner software project and populates it from the supplied retirement-planner-github.zip package. The repository is distinct from the Retirement Planning Professional™ Library documentation repository.

Recommended method: Create an empty private repository on GitHub, clone it to the computer, extract the ZIP to a temporary folder, copy the contents of the ZIP's retirement-planner folder into the cloned repository, validate the structure, and then make the first commit and push.

Why this method is preferred

- It preserves hidden files such as .gitignore and the .github folder.
- It avoids accidentally nesting retirement-planner inside another retirement-planner folder.
- It establishes the GitHub remote before the first project commit.
- It provides a clean, reviewable initial commit.
- It is safer than uploading dozens of files through the GitHub web interface.

2. What the ZIP Package Contains

The supplied ZIP contains one top-level folder named retirement-planner and 56 starter files. The package provides repository scaffolding rather than a completed Version 4 application. Several calculation and optimization modules are intentionally placeholders for later implementation.

Area	Purpose
src/retirement_planner	Python package with calculators, engines, models, optimizers, reporting, and utilities.
config	Model configuration and tax-year configuration placeholders.
data/reference	Annual tax, RMD, IRMAA, ACA, and other authoritative reference data.
data/scenarios	Household scenario input files.
docs	Architecture, workbook design, methodology, and user guidance.
scripts	Workbook generation and scenario execution entry points.
tests	Unit and integration tests.
workbooks/reference	Approved reference workbook files, including the repaired RC2 workbook.
.github	GitHub Actions, issue templates, and pull-request template.
outputs	Generated files; intentionally excluded from source control except .gitkeep.

Important baseline status

Do not treat the scaffold as a finished application. The current repository establishes architecture, naming, documentation, and test locations. The workbook generation script and many calculation modules still require the approved Version 3.1 baseline logic to be migrated and verified.

3. Prerequisites

- A GitHub account.
- Git installed on the computer. Git for Windows includes Git Bash and Git Credential Manager.
- An IDE or editor such as Visual Studio Code.
- The downloaded retirement-planner-github.zip file.
- Python 3.11 or later for validation and future development.
- The repaired Retirement_Planner_v3.1_RC2.xlsx workbook when it is ready to be added as the approved baseline.

Verify Git

```
git --version
```

A version number confirms that Git is installed. Configure your identity once if necessary:

```
git config --global user.name "YOUR NAME"
git config --global user.email "YOUR-EMAIL@example.com"
```

4. Create the Empty GitHub Repository

1. Sign in to GitHub and select New repository.
2. Choose the correct owner: your personal GitHub account or the organization that will own the project.
3. Enter the repository name retirement-planner.
4. Enter a description such as: Professional retirement-planning calculation engine and Excel workbook generator.
5. Select Private while the workbook, methodology, and code remain proprietary.
6. Do not initialize the repository with a README, .gitignore, or license. Those files already exist in the ZIP package.
7. Create the repository and leave the Quick Setup page open.

Why the repository must be empty: GitHub recommends leaving README, license, and .gitignore unselected when importing an existing local project. Doing so avoids an unnecessary merge conflict during the first push.

5. Clone the Empty Repository

Copy the HTTPS repository URL from GitHub. It will have this form:

```
https://github.com/YOUR-USERNAME/retirement-planner.git
```

Windows PowerShell

```
cd "C:\Users\YOUR-NAME\Documents\GitHub"
git clone https://github.com/YOUR-USERNAME/retirement-planner.git
cd retirement-planner
```

Git Bash

```
cd ~/Documents/GitHub
git clone https://github.com/YOUR-USERNAME/retirement-planner.git
cd retirement-planner
```

Because the GitHub repository is empty, Git may display a warning that the cloned repository is empty. That is expected.

Authentication: GitHub no longer accepts an account password for command-line Git operations. HTTPS normally uses Git Credential Manager, a browser sign-in, GitHub CLI, or a personal access token. SSH is also supported.

6. Extract the ZIP Without Creating a Nested Repository

Extract retirement-planner-github.zip to a temporary location such as Downloads. The extracted package should look like:

```
Downloads/
└── retirement-planner/
    ├── .github/
    ├── config/
    ├── data/
    ├── docs/
    ├── scripts/
    ├── src/
    ├── tests/
    ├── README.md
    └── ...
```

Critical: Copy the contents inside the extracted retirement-planner folder into the cloned retirement-planner folder. Do not copy the outer folder itself.

Correct result

```
Documents/GitHub/retirement-planner/
├── .git/              # created by git clone
├── .github/           # copied from ZIP
├── config/
├── data/
├── docs/
├── src/
├── tests/
├── README.md
└── pyproject.toml
```

Incorrect nested result

```
Documents/GitHub/retirement-planner/
└── retirement-planner/   # wrong extra level
    ├── src/
    └── README.md
```

7. Populate the Cloned Repository

The easiest method is to select all files and folders inside the extracted retirement-planner folder and copy them into the cloned repository folder. Allow Windows to merge folders if prompted.

Optional PowerShell copy command

```
$source = "$HOME\Downloads\retirement-planner\*"
$destination = "$HOME\Documents\GitHub\retirement-planner"
Copy-Item -Path $source -Destination $destination -Recurse -Force
```

PowerShell may not copy hidden items with a simple wildcard in every situation. Confirm that .github and .gitignore are present afterward. File Explorer or robocopy is safer for hidden files.

Recommended robocopy command

```
robocopy "$env:USERPROFILE\Downloads\retirement-planner" `
    "$env:USERPROFILE\Documents\GitHub\retirement-planner" `
    /E /COPY:DAT /R:2 /W:2
```

Do not use /MIR unless you understand its deletion behavior. The /E option copies all subdirectories, including empty ones.

8. Validate the Repository Structure

From the repository root, run:

```
git status
```

The copied project files should appear as untracked files. The .git directory should not appear because Git manages it internally.

Confirm the remote

```
git remote -v
```

Both fetch and push should point to the GitHub repository URL.

Confirm the top-level files

```
Get-ChildItem -Force     # PowerShell
# or
ls -la                   # Git Bash
```

Confirm that README.md, LICENSE, .gitignore, .github, src, tests, docs, and pyproject.toml are visible.

9. Create the Python Environment and Run Initial Tests

PowerShell

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
pytest
```

Git Bash

```
python -m venv .venv
source .venv/Scripts/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
pytest
```

Expected result: The starter federal-tax unit test and repository-structure integration test should pass. A passing test suite verifies the scaffold, not the accuracy of the full retirement-planning model.

10. Test the Workbook Builder Scaffold

```
python scripts/build_workbook.py
```

The current scaffold generates a placeholder workbook in outputs/excel. Because outputs is excluded by .gitignore, the generated file should not be committed. This test confirms that Python and openpyxl are functioning.

11. Add the Approved Version 3.1 Baseline Workbook

After the repository scaffold is working, copy the repaired workbook into:

```
workbooks/reference/Retirement_Planner_v3.1_RC2.xlsx
```

The workbook is small enough for ordinary Git tracking. Git LFS is not required at its current size. Before committing, verify that the file opens in desktop Excel without a recovery message.

Baseline rule: RC2 should be preserved as an immutable reference artifact. Future workbook builds should use new filenames and should not silently overwrite the approved baseline.

12. Make the Initial Commit

```
git status
git add .
```

```
git status
git commit -m "Initialize Retirement Planner repository scaffold"
```

Review the second git status before committing. Confirm that there are no passwords, personal financial data, temporary Excel files, virtual-environment files, or generated outputs in the staged list.

13. Push the Initial Population to GitHub

```
git branch -M main
git push -u origin main
```

The -u option records origin/main as the upstream branch. Future pushes from main can use git push without additional arguments.

14. Verify the GitHub Repository

8. Refresh the repository page in GitHub.
9. Confirm that README.md displays on the repository home page.
10. Open the Actions tab and verify that the Test Retirement Planner workflow runs.
11. Confirm the source directories, docs, configuration, tests, and issue templates are visible.
12. Confirm the repository remains Private.
13. Confirm no personal scenario data or credentials were committed.

15. Create the Development Branch

```
git checkout -b develop
git push -u origin develop
```

Use main for approved, release-ready work and develop for integrated work toward the next release. Create short-lived branches from develop for individual changes:

```
git checkout develop
git pull
git checkout -b fix/conversion-tax-funding-merge
```

16. Recommended GitHub Settings

- Keep the repository private during workbook and calculation-engine review.
- Protect main after the initial push.
- Require pull requests before merging into main.
- Require the test workflow to pass before merging.
- Disable force pushes and branch deletion for main.
- Enable Issues for calculation defects and feature requests.
- Use Releases for approved workbook and software packages rather than committing generated release ZIPs to source folders.

17. Normal Development Workflow

```
git checkout develop
git pull
git checkout -b feature/name-of-change

# Edit files and run tests
pytest

git add .
git commit -m "Describe the completed change"
git push -u origin feature/name-of-change
```

Open a pull request from the feature branch into develop. Move tested, documented work from develop into main through a release pull request.

18. Troubleshooting

Problem	Likely cause	Correction
Repository contains retirement-planner/retirement-planner	The outer ZIP folder was copied instead of its contents.	Move the inner contents up one level and delete the empty nested folder.
.github or .gitignore is missing	Hidden files were not copied.	Use File Explorer with hidden items enabled or robocopy /E.
Push rejected because remote has work	The GitHub repository was initialized with a README, license, or .gitignore.	For a brand-new repository, delete and recreate it empty, or pull and merge the remote commit carefully.
Authentication failed	A GitHub password was used for command-line Git.	Use Git Credential Manager/browser authentication, GitHub CLI, a personal access token, or SSH.
pytest cannot import retirement_planner	The package was not installed in editable mode or the virtual environment is inactive.	Activate .venv and run python -m pip install -e .
Excel temporary file appears in git status	The workbook is open and created a ~$ temporary file.	Close Excel; .gitignore should exclude ~$*.xlsx.
Generated workbook appears in Git	The output was placed outside outputs or .gitignore was changed.	Move generated files to outputs and restore the ignore rule.

19. Initial Population Completion Checklist

☐ Empty private GitHub repository created as retirement-planner.

☐ Repository cloned to the intended local development folder.

☐ ZIP contents copied without an extra nested retirement-planner folder.

☐ .github and .gitignore confirmed present.

- ☐ Git remote confirmed with git remote -v.
- ☐ Python virtual environment created and dependencies installed.
- ☐ pytest completed successfully.
- ☐ Placeholder workbook build tested.
- ☐ Repaired RC2 workbook added to workbooks/reference after verification.
- ☐ No credentials or personal financial data staged.
- ☐ Initial commit created and pushed to main.
- ☐ GitHub Actions workflow completed.
- ☐ develop branch created and pushed.
- ☐ main branch protection configured.

Appendix A - Repository Structure Included in the ZIP

```
retirement-planner/
.github
  ISSUE_TEMPLATE
    calculation-defect.md
    feature-request.md
  pull_request_template.md
  workflows
    test.yml
.gitignore
CHANGELOG.md
CONTRIBUTING.md
LICENSE
README.md
ROADMAP.md
config
  model.yml
  tax-year-2026.yml
data
  reference
    README.md
  scenarios
    reference-household.yml
docs
  architecture
    SYSTEM_ARCHITECTURE.md
  design
    WORKBOOK_SPECIFICATION.md
  methodology
    CALCULATION_METHODOLOGY.md
  user-guide
    GETTING_STARTED.md
examples
  README.md
outputs
  .gitkeep
  csv
  excel
  pdf
pyproject.toml
releases
  README.md
requirements.txt
scripts
  build_workbook.py
  run_scenario.py
src
  retirement_planner
    __init__.py
    calculators
      __init__.py
      aca_ptc.py
      federal_tax.py
      healthcare.py
      irmaa.py
```

```
        portfolio.py
        rmd.py
        social_security.py
      engines
        __init__.py
        projection_engine.py
        scenario_engine.py
      models
        __init__.py
        assumptions.py
        household.py
        results.py
      optimizers
        __init__.py
        roth_conversion.py
        social_security_claiming.py
        withdrawal_sequence.py
      reporting
        __init__.py
        csv_exporter.py
        excel_builder.py
        pdf_report.py
      utils
        __init__.py
        config_loader.py
        validation.py
templates
tests
  calculators
    test_federal_tax.py
  integration
    test_repository_structure.py
workbooks
  reference
    README.md
```

Appendix B - Condensed PowerShell Command Sequence

```
# 1. Clone the empty GitHub repository
cd "$HOME\Documents\GitHub"
git clone https://github.com/YOUR-USERNAME/retirement-planner.git
cd retirement-planner

# 2. Copy the extracted ZIP contents into this folder using File Explorer
#    or robocopy from another PowerShell window.

# 3. Verify repository and files
git remote -v
Get-ChildItem -Force
git status

# 4. Set up Python
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

```
python -m pip install -r requirements.txt
python -m pip install -e .
pytest

# 5. Test workbook generation
python scripts/build_workbook.py

# 6. Commit and push
git add .
git status
git commit -m "Initialize Retirement Planner repository scaffold"
git branch -M main
git push -u origin main

# 7. Create development branch
git checkout -b develop
git push -u origin develop
```

Appendix C - Authoritative References

GitHub documentation consulted for the procedures in this guide:

- Creating a new repository: https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository
- Cloning a repository: https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository
- Adding locally hosted code to GitHub: https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github
- About authentication to GitHub: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github
- Managing remote repositories: https://docs.github.com/en/get-started/git-basics/managing-remote-repositories